Relatório do Trabalho Teórico/Computacional
Cadeias de Markov

Ruan Pereira Alves – 569551

9 de setembro de 2026

1 Tarefa 01

Dado que recebemos a matriz de probabilidade de transição, podemos demonstrar o grafo direcionado com as probabilidades a seguir:

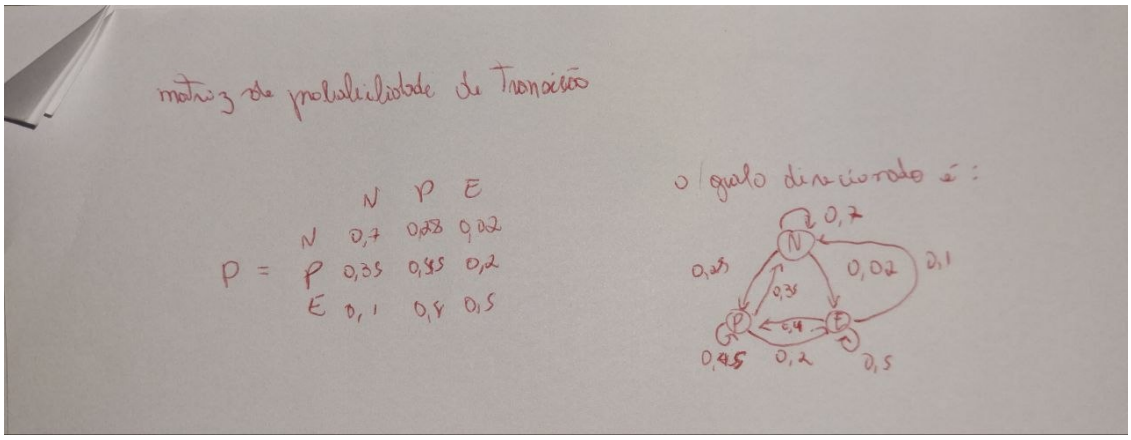


Figura 1: Grafo direcionado com as probabilidades.

Com as possíveis transições de estado.

2 Tarefa 02

Para a segunda tarefa, foi feito um jupyter notebook com o desenvolvimento pedido em cada um dos itens, que serão explicados detalhadamente abaixo:

2.1 item i

Uma das condições para que a cadeia de Markov seja definida é que seja um processo estocástico que satisfaz a propriedade markoviana, que nos diz que as probabilidades de

todos os estados futuros dependem somente do estado atual. Outra das condições é que ela seja homogênea, o que implica que a a cadeia é homogênea caso ela possa ter n passos de transição.

De forma geral, podemos definir a transição entre dois estados quaisquer a partir da equação de Chapman-Kolmogorov:

$$p_{ij}(m+n) = \sum_{k} p_{ik}(m) \cdot p_{kj}(n)$$

Podemos assim definir a matriz de probabilidade de transição para n passos, representada por P(n), da seguinte forma:

$$\mathbf{P}(n) = \mathbf{P}^n = \prod_{i=1}^{n} \mathbf{P}$$

que foi a forma escolhida para o programa. Considerando n = 10, 20, 50, 100, 200. Assim, obtivemos:

```
•[3]: #matriz de probabilidade de transição de 1 passo
P = np.array([
    [0.7, 0.28, 0.02],
    [0.35, 0.45, 0.2],
    [0.1, 0.4, 0.5]
])

#definindo N
N = np.array([10,20,50,100,200], dtype=int)

# realizando a operação pela propriedade de exponencial
for i in N:
    P_N = np.linalg.matrix_power(P,i)
    print("resultado para ",i)
    print(P_N)
```

```
[[0.7  0.28 0.02]
 [0.35 0.45 0.2 ]
 [0.1  0.4  0.5 ]]
resultado para  10
[[0.47640788 0.36077959 0.16281253]
 [0.47526051 0.36106138 0.1636781 ]
 [0.47405832 0.36135664 0.16458504]]
resultado para  20
[[0.4756114  0.36097521 0.1634134 ]
 [0.47560904 0.36097579 0.16341518]
 [0.47560657 0.36097639 0.16341704]]
resultado para  50
[[0.47560976 0.36097561 0.16341463]
 [0.47560976 0.36097561 0.16341463]
 [0.47560976 0.36097561 0.16341463]]
resultado para  100
[[0.47560976 0.36097561 0.16341463]
 [0.47560976 0.36097561 0.16341463]
 [0.47560976 0.36097561 0.16341463]]
resultado para  200
[[0.47560976 0.36097561 0.16341463]
 [0.47560976 0.36097561 0.16341463]
 [0.47560976 0.36097561 0.16341463]]
```

Figura 2: Valores encontrados para cada valor de N.

2.2 item ii

Como podemos observar no item anterior, podemos observar que a partir de n = 50, os valores atingem convergência, ou seja a matriz de probabilidade de se aproxima de uma matriz na qual as linhas tem a mesma distribuição de probabilidade, sendo chamadas de probabilidade de estado estacionário.

2.3 item iii

Para o terceiro item, foi considerado o estado inicial primeiro para N, depois P e por último E. Sendo assim, cada estado inicial se torna um vetor, e assim podemos realizar a probabilidade dos estados com base no vetor inicial como:

$$\mathbf{p}(n) = \mathbf{p}(0)\mathbf{P}(n) = \mathbf{p}(0) \cdot \mathbf{P}^n$$

que foi a propriedade utilizada no código para realizar os cálculos. Temos assim como resultado:

```python
#definindo os estados iniciais na ordem N, P, E como vetores

estados_iniciais = {
    "Iniciando em N [1, 0, 0]": np.array([1.0, 0.0, 0.0]),
    "Iniciando em P [0, 1, 0]": np.array([0.0, 1.0, 0.0]),
    "Iniciando em E [0, 0, 1]": np.array([0.0, 0.0, 1.0])
}

# para cada nome de vetor e vetor
for nome, pi_0 in estados_iniciais.items():
    print(f"Cenario: {nome}")
    for n in N:
        # multiplicando a matriz de transição com n passos pelo vetor do estado inicial
        pi_n = pi_0 @ np.linalg.matrix_power(P, n)
        print(f"  n = {n:3d} | Probabilidade de cada estado [N, P, E]: {pi_n}")
```

```
Cenario: Iniciando em N [1, 0, 0]
  n =  10 | Probabilidade de cada estado [N, P, E]: [0.47640788 0.36077959 0.16281253]
  n =  20 | Probabilidade de cada estado [N, P, E]: [0.4756114  0.36097521 0.1634134 ]
  n =  50 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
  n = 100 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
  n = 200 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
Cenario: Iniciando em P [0, 1, 0]
  n =  10 | Probabilidade de cada estado [N, P, E]: [0.47526051 0.36106138 0.1636781 ]
  n =  20 | Probabilidade de cada estado [N, P, E]: [0.47560904 0.36097579 0.16341518]
  n =  50 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
  n = 100 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
  n = 200 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
Cenario: Iniciando em E [0, 0, 1]
  n =  10 | Probabilidade de cada estado [N, P, E]: [0.47405832 0.36135664 0.16458504]
  n =  20 | Probabilidade de cada estado [N, P, E]: [0.47560657 0.36097639 0.16341704]
  n =  50 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
  n = 100 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
  n = 200 | Probabilidade de cada estado [N, P, E]: [0.47560976 0.36097561 0.16341463]
```

Figura 3: Valores das matrizes de acordo com o vetor inicial.

2.4 item iv

Pelos itens anteriores, podemos observar que o estado com maior probabilidade estacionária é o estado N (trânsito normal), que atinge 47,52%. Isso acontece pois, devido ao processo ser estacionário, as probabilidades independem do instante inicial. Podemos observar melhor graficamente:

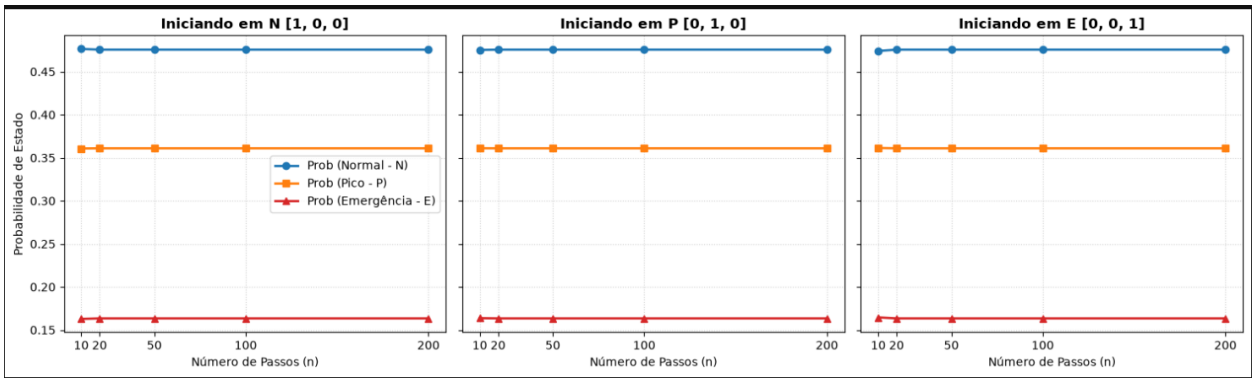


Figura 4: Gráfico ilustrando as probabilidades de acordo com cada cenário inicial.

2.5 item v